Week 5: When Algorithms Choose Who Gets Help

Search, Consolidated, and the Choices Upstream of the Algorithm

FIT1061: Introduction to Artificial Intelligence

2026-08-25

The Story: Mapping Haiti, January 2010

On 12 January 2010, a magnitude 7.0 earthquake struck near Port-au-Prince, the capital of Haiti. Buildings across the city collapsed. Roads were blocked by rubble, and some were destroyed completely. Aid was arriving (trucks of food, water, and medical supplies) but the people directing those trucks faced a basic problem: **they did not have a good map.** Before the quake, Port-au-Prince was barely mapped at all. OpenStreetMap, the free and open map of the world, held only a portion of Haiti's road network. The major commercial maps had even less. To route aid past a destroyed road, you first have to know that the road is destroyed, and where the open roads are.

What happened next is one of the clearest examples of people and software solving a problem together. Satellite companies released high-resolution images of Port-au-Prince, taken before and after the quake. Volunteers around the world opened those images on their own computers and began **tracing** what they saw (roads, buildings, and damage) directly into OpenStreetMap. Nobody had to be in Haiti to help. You needed the images, a keyboard, and an internet connection.

The progress was rapid. About **53 hours** after the earthquake, one of the project's coordinators reported that more than **400 editing sessions** had already improved the map. Over the following month, close to **600 contributors** kept building. The result, in the affected urban areas, held **more road detail than the United Nations' own dataset and Google Maps.** A map built by volunteers, from satellite images, in weeks, beat the official sources: a map built, in effect, **from almost nothing.**

That effort helped launch a **movement**. The **Humanitarian OpenStreetMap Team (HOT)** was founded in the wake of the Haiti response; with projects like **Missing Maps** (run with the Red Cross and Médecins Sans Frontières) it now crowd-builds road and building maps *before* and during crises. And once a map exists, other software routes over it: the **World**

Food Programme's "Route The Meals" plans aid-delivery truck routes. Building the map, and routing over it, are two different jobs done by two different sets of hands.

Here is the quiet point this week turns on. Once you use a map to route aid, the routing decides **who gets help first**. A search algorithm running over that map is not just solving a puzzle; it is ranking human need. But the algorithm itself is neutral machinery. The choices that carry the values are made *before* it runs, and there are three of them: **what you map** (what you record about the world, not the terrain itself), **what you tell the search to follow** (the heuristic), and **what you measure** as success (the objective). This week we consolidate the three search tools you already have, and then walk those three choices in turn.

Consolidating the Search Toolkit

For the past few weeks, almost every search asked the same question: *there is a start and a goal, so what is the path between them?* You built **three** different ways to answer it. They are tools, and the tool itself is neutral: the same search runs the same way whoever sets it going.

Three searches, one job

> **i** Recall: the three path-finding searches
>
> - **Breadth-first search** explores the graph in **layers**, one ring of neighbours at a time. Its waiting list is a **queue** (first in, first out). It guarantees the path with the **fewest edges**. It trades **memory** for that **guarantee**: it must hold a whole layer at once.
> - **Depth-first search** dives **deep** along one branch before backing up. Its waiting list is a **stack** (last in, first out). It finds *a* path quickly, but gives **no** guarantee that the path is shortest. It trades the **guarantee** for **memory**.
> - **Greedy best-first search** follows a **heuristic** (an estimate of how far each place is from the goal) and always expands whatever *looks* closest. It is fast, but the estimate can **mislead** it down a longer route. It trades **guarantees** for **speed**.

You rarely reach for one of these by name. You reach for the **trade-off** you can afford.

Which search fits, and when?

Put the searches on the same map. Picture the Haiti situation as a graph: a supply **depot** is the start, the **hospital** (F) is the goal, lettered **areas** sit between them, the lines are **surviving roads**, and a destroyed road is simply a **missing edge**. Greedy also carries an **estimate** at

each area — the straight-line distance still to go (lower *looks* closer). Run two searches from the depot to F:

- **Breadth-first** ignores the estimate and sweeps outward layer by layer. It finds a **fewest-edges** route: **D → B → F (2 edges)**.
- **Greedy best-first** follows the estimate — and A *looks* closest (estimate 2). But the direct road **A → F is destroyed**, so greedy detours **D → A → C → F (3 edges)**. It trusted the estimate straight into the longer way.

Same map, same estimates — **the map decided the route.** The estimate was honest about *direction* but blind to which roads survived; breadth-first, ignoring it, still found the shorter route *here*. Neither is "best" in general:

Algorithm	Waiting list	Explores	Shortest route?
Breadth-first	queue (first in, first out)	wide, in layers	**fewest edges**
Depth-first	stack (last in, first out)	deep, one branch	**no**
Greedy best-first	priority, by estimate	toward the estimate	**not guaranteed**

! Key idea: the map decides which search fits

No one algorithm is best on every map. What the graph records — which roads survive, what the estimate is — decides which search finds the good route. Choosing the search, and **building the graph it runs on**, is part of solving the problem.

Two kinds of problem

Step back further, and something even more useful appears. Every problem in Weeks 2 to 4 was one of just **two kinds**.

- **Path-finding.** There is a start and a goal, and you want the **route** between them. Breadth-first, depth-first, and greedy best-first all solve this kind.
- **Optimisation (local search).** There is **no goal state** to reach. Instead you hold one **candidate** answer and improve it, step by step, until you cannot improve it further. Hill climbing on the 8-queens board, from last week, is this kind: you keep one board and move a queen to make it better.

The very first question to ask about any new problem is: **which of these two kinds is it?** A path-finding problem calls for a search that explores a graph; an optimisation problem calls for a method that improves a single candidate. Naming the kind first saves you from forcing the wrong tool onto the problem.

(One reliability trick from the optimisation family is worth a single line: if hill climbing gets **stuck** where no small move improves, you can **re-run it from a fresh random start and keep the best result**: a "random restart." That, and comparing the three path-finding searches across different graph shapes, are the extensions to reach for if you are aiming beyond a Pass.)

What We Map Is a Choice

Be careful with the word "map." The **terrain** is real and given: the earthquake happened, the rubble fell, some roads are open and some are blocked. Nobody *chooses* the terrain. But the **map** is not the terrain — it is a *model* of it, and someone decides what the model contains: which roads get traced in, what is marked passable or destroyed, what is left blank. A search can only ever route over the **map**, never the terrain directly. So the choice is not *the world*; it is **what we record about the world.**

That is not a hypothetical. Before the quake, Port-au-Prince was barely mapped; the graph that guided the aid was built, under pressure, by around 600 volunteers who happened to have satellite imagery, a keyboard, and connectivity. Every road they traced went into the map. Every road they missed did not. The search that ran afterwards could only help people who were **on the map**. The gaps were not in the terrain — they were in what got recorded, so a place left off the map was one the routing could never reach.

So the first value-laden choice is made before any search runs: **what we put on the map.** Keep this in mind; we return to it when we ask whose voice was missing. (The map's *shape* matters too — it decides which of breadth-first, depth-first, or greedy is efficient — but that is a question of speed; *what we record* is the question of who is even visible.)

Maps have carried this kind of power for a long time. Colonial-era maps left whole inhabited regions blank because the mapmakers did not count what was there; in the mid-twentieth century, banks in the United States drew maps that shaded certain neighbourhoods as "risky," and people in those areas were then refused loans for decades — a practice now called *redlining*. In each case the map was treated as neutral fact, but its contents were a choice, and the choice decided who was served. The Haiti crisis map is a hopeful version of the same pattern: built fast, to help — but still shaped by who held the keyboard. A later, quieter point is worth keeping in view too: even the *risk numbers* on our map are not facts but **estimates** — somebody scored each cell, from a photo, under pressure, without ground truth.

What We Follow Is a Choice

Now a concrete map — a **maze**. A convoy must get from a supply **depot** on the left to a **hospital** on the right, but an **impassable peak** (collapsed buildings, blocked streets) sits in

the middle. Around that peak the ground is graded: an **inner ring of high risk** (close to the danger), a wider **outer ring of low risk**, and clear ground beyond. On a single route, two things trade off:

- **distance travelled** — the **number of steps** the convoy drives. **Lower is better** (speed to the patients waiting).
- **risk / exposure** — **add up the risk of every cell the route crosses** (a clear cell scores **0**, a low-risk ring cell **1**, a high-risk ring cell **3**). **Lower is better** (danger to the convoy).

So each route turns into exactly **two numbers** — a *distance* and a *risk* — which we can read as a single **point** on a plane with distance across and risk up. (Speed-first, for instance, is 20 steps through the high-risk cells: the point **(20, 25)**.)

The convoy can **hug the peak** (short, but through the high-risk ring) or **swing wide around it** (safe, but long). Which route does greedy best-first find? That depends on its **heuristic**. Greedy expands whatever *looks* best, scoring each step by

$$h = \text{distance still to go} \; + \; \lambda \cdot \text{risk here}$$

The number λ is the **safety weighting**: how much distance you are willing to spend to avoid one unit of risk. And λ **is your choice.** Run greedy three times, with three defensible weightings:

Heuristic	What it does	Distance	Risk
speed-first ($\lambda = 0$)	ignore risk; hug the peak	20	25
balanced ($\lambda = 1$)	some caution; arc through the low-risk ring	22	12
safety-first ($\lambda = 4$)	avoid the danger; swing all the way around	26	0

Same map, same greedy machinery. Change only *how heavily it weights safety*, and it builds a **different route** through the maze. Less risk costs more distance. The heuristic is the choice.

i Where the route flips: a worked lambda trace

At the key junction the convoy can go **through** (2 steps closer to the goal, but crossing risk 3) or **detour** (4 steps closer, crossing risk 1). Greedy compares h = distance-to-go$+\lambda\cdot$risk:

- through: $h = 10 + 3\lambda$

- detour: $h = 12 + 1\lambda$

They tie when $10 + 3\lambda = 12 + \lambda$, i.e. $\lambda = 1$. Below $\lambda = 1$ greedy goes *through*; above it, it *detours*. So a single number — the weighting you chose — is exactly what flips the route.

The Pareto Frontier

Those three routes are not the only ones. In fact you could score **every possible route** as a point — a whole **cloud** of plans — and most of them are **beaten on both counts** at once by one of the three above. Run **depth-first search**, for instance, with no strategy at all, and it finds *a* route — but a wasteful one: it wanders through much of the maze, **76 steps** long and still crossing **risk 49**. Line the routes up, each as a point:

Route	Found by	Distance	Risk
speed-first	greedy, $\lambda = 0$	20	25
balanced	greedy, $\lambda = 1$	22	12
safety-first	greedy, $\lambda = 4$	26	0
DFS	depth-first, no strategy	76	49

Look at the DFS route: **76 steps *and* risk 49.** Every greedy route is shorter *and* safer. There is no reason ever to choose DFS; it is beaten on both counts at once. That pattern has a name, from the economist **Vilfredo Pareto**, who around 1900 studied how resources spread across a society.

i Definition: dominance and the Pareto frontier

Option A **dominates** option B if A is *at least as good* as B on **every** axis, and *strictly better* on **at least one** axis.
An option is **Pareto-optimal** if **nothing dominates it**.
The set of all Pareto-optimal options is the **Pareto frontier**: the outer edge of what is achievable. You can move *along* it, trading one axis for the other, but you cannot push *past* it.

Finding the frontier

To find the frontier, take each route in turn and ask: *is there another route that is no worse on either axis and strictly better on at least one?* If yes, it is dominated: rule it out. If no such route exists, it survives.

- **DFS** (76, 49) is dominated by every greedy route — each is both shorter and safer. Rule it out.
- **speed-first, balanced, and safety-first** are each mutually non-dominated: each trades distance for risk in a way the others do not. None is beaten on both counts.

Those three survive; they are the Pareto frontier, and they are exactly the routes our three defensible weightings produced. Speed-first is shortest but riskiest; safety-first is safest but longest; balanced sits between. Moving along the frontier, you **buy less risk by spending more distance**. Each is a genuine, defensible trade-off.

> ! What the frontier does, and does not do
>
> The Pareto frontier tells you the trade-offs you **cannot beat**: any route off the frontier can be improved on both axes at once, so it is never worth choosing.
> The frontier does **not** tell you **which** route to pick. Speed-first, balanced, and safety-first are all on it, and the maths alone cannot rank them. Choosing between them is the next question, and the maths cannot answer it.

You have seen this frontier before

This shape is everywhere once you look. Your navigation app offers "fastest," "fuel-saving," and "fewer highways" — three points on a frontier, and by tapping one you pick a weighting, exactly as you picked a λ. The same structure returns later in this unit with far higher stakes: when a system decides who gets flagged for review, the two axes become **false alarms** (people wrongly flagged) and **missed cases** (real cases let through). You cannot drive both to zero at once; there is a frontier, and someone chooses the point — only now the points are decisions about **people**. The lesson travels: the maths hands you the trade-offs you cannot beat, and a human always picks which one to live with.

At scale: when you cannot eyeball it

The small map made the idea clean, almost too clean — you could see the answer. Real cities are not clean. Picture a bigger map: whole **no-go districts** (collapsed or flooded blocks you cannot enter) and **risk zones of different intensity**, arranged so that *every* route from the depot to the hospital must cross some risk. Now try the obvious rules and watch each one fail in its own way:

- **Distance-only** (ignore risk): drives straight through the most dangerous zone. Fast, reckless.
- **Forbid all risk** (never enter a risky cell): finds **no route at all** — if risk blocks every crossing, a "zero-risk only" rule is simply impossible. You cannot forbid risk here; you can only *trade* it.

- **Risk-only** (ignore distance): takes the safest crossing, but crawls the long way round — and it still is not zero, because there is no risk-free path.
- **Distance + λ·risk** (the weighted rule from before): threads a sensible line, and the weight λ lets you slide between fast-and-risky and slow-and-safe.

The lesson the small map hinted at becomes unavoidable at scale: the naive rules each break, and the only workable approach is to **name what you are trading and tune it deliberately.** That tuning is a value choice — which is exactly the question the next section turns to.

What We Measure Is a Choice

Picking a route means picking a point on the frontier, and that means deciding **which axis matters more** — a decision the search cannot make for you.

- "**distance**" is a stand-in for **speed to the patients waiting**.
- "**risk**" is a stand-in for **danger to the convoy** crossing the hazard.

The safety weighting λ is exactly this value choice, and you set it *before* the search runs a single step. Set λ low and the search will drive people through danger to save time; set it high and it will keep the convoy safe at the cost of a longer wait. The optimisation is entirely **downstream** of that choice. The search is only ever as fair as the weighting you decided to give it.

This is really a choice between **safety and reach**. Imagine two aid coordinators, each looking at the same frontier.

- **Reach first.** Send the convoy through, because people are waiting. The route is fast, but the crew drives into danger.
- **Safety first.** Route around, because you will not drive people into a hazard. The crew is safe, but the patients wait longer.

Both are choosing routes **on the frontier**, and the maths ranks neither above the other. Which one to send is a **value judgement** the search cannot settle: the moment you write down the single number you will minimise, you have made the choice yourself.

And there is a sharper question hiding inside it: **who bears the risk?** The person who *sets* the safety weighting is usually not the person who *carries its cost.* The weighting might be chosen by whoever runs the operation, often far from the road; the danger falls on the crew in the convoy, and the delay falls on the patients who wait. A single number, set once, silently distributes risk and delay across people who never touched the dial. *Should whoever sets the weighting be the one who bears it?*

Discussion: whose voice is missing?

Return to the Haiti map and ask three questions, one for each choice:

- **What went on the map** — who traced it, who marked what counts as "hazardous", and who was far from a keyboard, an internet connection, or a satellite image?
- **What the search followed** — who set how heavily it should weigh risk?
- **What we measured** as success — who chose speed over safety, and who bears each?

A map built to guide aid still reflects the **priorities of whoever drew it.**

That first question is not abstract. The Haiti map was built by around 600 volunteers who had satellite imagery, a keyboard, and connectivity. The people *least* likely to be represented in such a map are exactly those **without** digital access, who are often also among those in greatest need. The map was a genuine humanitarian achievement, and it carried a bias built into how it was made.

Finally, a harder question, which these notes will **pose but not answer**. We measured "risk" as exposure to the hazard — the same for everyone on the route. But some people are far more **vulnerable** to that exposure: children, the elderly, people with medical needs. Should the risk axis **weight them more heavily**? Would the frontier move? Would a different route win? This is the kind of question you will be asked to reason about yourself, and it is deliberately left open here. It is worth holding onto, because this tension between what is easy to measure and what actually matters returns later in the unit, when machines begin scoring people directly.

Summary

Key vocabulary:

Term	Meaning
Path-finding	A problem with a start and a goal, where you want the route between them
Optimisation (local search)	A problem with no goal state; you hold one candidate and improve it
Heuristic	An estimate used to guide a greedy search: it encodes what you treat as "best"
Dominance	Option A dominates B if A is at least as good on every axis and strictly better on at least one
Pareto-optimal	An option that nothing dominates

Term	Meaning
Pareto frontier	The set of all Pareto-optimal options: the trade-offs you cannot beat
Objective / axis	The measure a search optimises; choosing it encodes values

Key takeaways: the algorithm is neutral machinery; the values live in three choices upstream of it.

- **What we map** is a choice. Search routes over the map we build, not the terrain itself; on the Haiti map, the gaps were whoever was far from a keyboard. (The map's *shape* also decides which of breadth-first, depth-first, or greedy fits — but that is speed, not visibility.)
- **What we follow** is a choice. How heavily we weight safety (the heuristic) builds a different route over the same map.
- **What we measure** is a choice. The Pareto frontier keeps every route we cannot beat on all measures at once, but does **not** tell you which to pick. Which one to pick is a value judgement the maths cannot make for you.

What's next: we turn from *finding* answers to *judging under uncertainty*: how a machine can reason about what is probably true when it cannot be sure.

In the Lab

This week: routing aid on the humanitarian network, and finding the frontier.

1. **Run** breadth-first, depth-first, and greedy best-first on the humanitarian network, routing from the depot to a target area.
2. **Compare greedy under two distance estimates** (Euclidean vs Manhattan) to see how the estimate you pick changes the route it builds.
3. **Find the Pareto frontier** over the four candidate target areas — trading off path cost against population served: list the candidates, rule out the dominated ones, and report which survive.
4. **Reflect** on the axes you were given: which axis did you choose to prioritise, and *why*? What does your choice say about whose need you counted first? This reasoning is yours to write; there is no single correct answer.

Aiming higher: any of these push past the plain Pass build.

- **Different graph structures.** Compare the three searches across maps of different shapes (wide and shallow, long and thin, densely connected) and explain which search fits which structure, and why.

- **Random restarts.** Take a local search that gets stuck, re-run it from fresh random starting points, and keep the best result. Measure how reliability improves as you allow more restarts.
- **A cheaper, smarter estimate.** Greedy trusts its estimate completely and can be misled. What could a **cheap, never-pessimistic** estimate of the remaining cost look like (one that never claims a place is *further* than it really is) and how would using it change which route the search returns?